Firebase Integration — Lava

This document describes how Firebase is integrated into Lava (Android client + lava-api-go service) and how to operate the distribute / stats workflows.

> **Sensitive values are NEVER documented here.** All real tokens, project IDs, app IDs, and tester emails live in `.env` (gitignored). The placeholders in `.env.example` are illustrative only.

Services enabled

Service	Where	Purpose
Crashlytics	Android client	Fatal + non-fatal exception reporting; custom keys for build_type, version_name, version_code, application_id
Analytics	Android client	User-visible flow events: login, search, browse, view-topic, download-torrent, provider-selected, endpoint-discovered
Performance Monitoring	Android client	Auto-instrumented HTTP traces; custom traces wired via `Trace.startTrace()`
App Distribution	Android (debug + release APKs)	Tester invitations driven by `.env` roster

Setup (one-time)

1. **Authenticate Firebase CLI**: `firebase login:ci` — copy the printed token into `.env` as `LAVA_FIREBASE_TOKEN`.
2. **Create a Firebase project + apps** (already done for Lava; project ID = `lava-vasic-digital`). For a fresh fork:

   ```
   firebase projects:create your-project-id --display-name "Your App"
   firebase apps:create ANDROID --project your-project-id --package-name your.package.name
   firebase apps:create WEB --project your-project-id "Your Web App"
   ```

   Copy the resulting app IDs into `.env` under `LAVA_FIREBASE_ANDROID_APP_ID` and `LAVA_FIREBASE_API_GO_APP_ID`.
3. **Refresh local config files**:

   ```
   ./scripts/firebase-setup.sh
   ```

This re-fetches `app/google-services.json` (Android) and `lava-api-go/firebase-web-config.json` (Web). Both are gitignored.

4. **Add tester emails to `.env`**: `LAVA_FIREBASE_TESTERS_OWNER`, `LAVA_FIREBASE_TESTERS_DEVELOPER`, `LAVA_FIREBASE_TESTERS_TESTER`. The list is loaded fresh each time `firebase-distribute.sh` runs.

Android client wiring

- **Plugins** applied via `buildSrc/.../AndroidApplicationConventionPlugin.kt`:
  - `com.google.gms.google-services`
  - `com.google.firebase.crashlytics`
- **Dependencies** in `gradle/libs.versions.toml`:
  - `firebase-bom` (33.15.0) — version-aligns the SDKs below
  - `firebase-analytics`
  - `firebase-crashlytics`
  - `firebase-perf`
- **Bootstrap** in `app/src/main/kotlin/digital/vasic/lava/client/LavaApplication.kt`:
  - `FirebaseApp.initializeApp(this)` — must be first to use the SDKs
  - Crashlytics: collection enabled in release builds only; custom keys (build_type, version_name, version_code, application_id) attached
  - Analytics: collection enabled in release builds only; user properties (build_type, app_version) set
  - Performance: collection enabled in release builds only
- **Tracker abstraction** at `core/common/src/main/kotlin/lava/common/analytics/AnalyticsTracker.kt` — Hilt-injectable interface with canonical event names. Features depend on this interface; the Firebase implementation lives in `:app` (`digital.vasic.lava.client.firebase.FirebaseAnalyticsTracker`) and is bound via `FirebaseModule.kt`. This keeps the Firebase coupling in `:app` per the Decoupled Reusable Architecture rule — features remain reusable.

Distribution workflow

The canonical operator distribute flow (replaces the old `releases/`-only delivery):

```
# Rebuild + distribute everything
./scripts/distribute.sh

# Variants
./scripts/distribute.sh --no-rebuild        # use existing
        artifacts
./scripts/distribute.sh --android-only      # rebuild +
        distribute Android only
./scripts/distribute.sh --release-only      # only the release-
        signed APK
./scripts/distribute.sh --debug-only        # only the debug APK
```

Internally:

1. `./build_and_release.sh` rebuilds all artifacts into `releases/<version>/`.

2. `./scripts/firebase-distribute.sh` uploads the Android APKs to Firebase App Distribution and notifies testers from `.env`.
3. `./scripts/firebase-stats.sh` prints the dashboard URLs the operator uses to inspect crashes / events / performance.

Anti-bluff posture: every script uses `set -euo pipefail`. There is no `|| echo WARN` swallowing of failures. A failed APK upload propagates as a non-zero exit, breaking the distribute chain.

Stats / observability

Live dashboards (the URLs are project-public; the data is access-controlled):

- Crashlytics: `https://console.firebase.google.com/project/<PROJECT_ID>/crashlytics`
- Analytics: `https://console.firebase.google.com/project/<PROJECT_ID>/analytics`
- Performance: `https://console.firebase.google.com/project/<PROJECT_ID>/performance`
- App Distribution: `https://console.firebase.google.com/project/<PROJECT_ID>/appdistribution`

Quick CLI: `./scripts/firebase-stats.sh` (`--days N` window, `--json` machine-readable).

For deeper queries, the lava-api-go service will expose `/admin/firebase-stats` (via the Firebase Admin SDK) — see `lava-api-go/internal/firebase/`.

Sensitive files (NEVER commit)

`.gitignore` lists:

- `.env`, `.env.*`, `.env.local`
- `app/google-services.json`, `**/google-services.json`
- `lava-api-go/firebase-web-config.json`
- `lava-api-go/firebase-admin-key.json`, `firebase-admin-key.json`, `**/firebase-admin-*.json`
- `firebase-debug.log`, `**/firebase-debug.log`
- `.firebase/`
- `firebase-distribute-*.log`

Pre-push hook + `scripts/check-constitution.sh` reject pushes that introduce these files (constitutional 6.H clause 5).

Rotation

If `LAVA_FIREBASE_TOKEN` is suspected leaked:

1. Revoke at https://myaccount.google.com/permissions (search "Firebase CLI").
2. `firebase login:ci` to mint a new one.
3. Replace in `.env`. Do not commit.

If `firebase-admin-key.json` is suspected leaked: revoke the service account in the Firebase Console → Project Settings → Service Accounts and create a new one. Replace in `lava-api-go/firebase-admin-key.json`. Do not commit.

Testing posture (anti-bluff)

Per constitutional clauses §6.J + §6.L (Anti-Bluff Functional Reality Mandate, invoked TEN TIMES on this project), Firebase tests MUST verify real reporting, not mocked calls:

- `app/src/androidTest/.../FirebaseIntegrationTest.kt` — installs the debug APK, triggers a known non-fatal, asserts the Crashlytics SDK accepted the report (collection enabled, no error from `recordException`).
- `app/src/test/.../FirebaseAnalyticsTrackerTest.kt` — verifies `AnalyticsTracker` calls Firebase SDK with the canonical event names + parameters.
- Falsifiability rehearsal recorded in commit body per §6.N: deliberately break `event()` (e.g., return without calling `analytics.logEvent`) → assert the test fails.

A Challenge Test (C9 onward) for end-to-end Firebase visibility — an operator-attested flow where a deliberate non-fatal in the debug build appears in the Crashlytics dashboard within 60 seconds — is the load-bearing acceptance gate.